



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Tarea Práctica #6: Algoritmo de ranqueo (Equipo)”

Sistemas Distribuidos y Paralelos

M.I.I. Carlos Adrián Pérez Cortez

Grupo: 004

Hora: N5

Salón: 4207

Equipo 9

MATRICULA	NOMBRE	PE
1952947	Heidi Pamela Martínez Martínez	ITS
1845517	Andrea Monserrat Méndez Flores	ITS
2109147	Alexia Hernandez Andrade	ITS
2004428	Alejandro Sauqué Pérez	ITS
2077884	Melida Krystal Mendoza Ventura	ITS

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 10 de
noviembre de 2024

Contenido

OBJETIVO	3
DIFICULTADES ENCONTRADAS	3
CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA (CLIENTE Y SERVIDOR)	4
CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA (CLIENTE Y SERVIDOR)	13
Conclusiones	14

OBJETIVO

Generar un algoritmo de ranqueo en base a los datos estáticos y dinámicos extraídos. Esto servirá para determinar cuál de las computadoras conectadas al servidor tiene la mejor conexión y características.

DIFICULTADES ENCONTRADAS

Tuvimos problemas con la separación de procesos para que la actualización de la tabla en el cliente y el envío de datos al servidor no interfirieran entre sí, además de asegurar que los datos correctos se mostraran en la interfaz del cliente, ya que hubo problemas donde solo se mostraba el núcleo y el ID.

Hubo problemas con la interrupción en hilos. En algunos casos el hilo encargado de enviar datos al servidor no se detuvo y el cliente no pudo estar enviando datos correctamente, o cerraba la conexión sin avisar al servidor.

Errores en la serialización / deserialización de objetos con `ObjectOutputStream` y `ObjectInputStream` para enviar objetos de tipo `Map<String, Object>`.

CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA (CLIENTE Y SERVIDOR)

SERVIDOR

```
1 package Ej;
2
3 import javax.swing.*;
4 import javax.swing.table.DefaultTableModel;
5 import javax.swing.table.TableRowSorter;
6 import java.awt.*;
7 import java.io.*;
8 import java.net.*;
9 import java.util.*;
10 import java.util.concurrent.*;
11 import java.util.List;
12 import java.io.File;
13
14 import oshi.SystemInfo;
15 import oshi.hardware.CentralProcessor;
16 import oshi.hardware.GlobalMemory;
17 import oshi.hardware.HardwareAbstractionLayer;
18 import oshi.hardware.HWDiskStore;
19 import oshi.hardware.NetworkIF;
20 import oshi.software.os.OperatingSystem;
21
22 public class Server {
23     private static final int UPDATE_INTERVAL = 5000; // 5 segundos
24     private static Map<String, ClientHandler> connectedClients = new ConcurrentHashMap<>();
25     private static DefaultTableModel model;
26     private static JTable table;
27     private static final int SERVER_PORT = 8080; // Puerto estático
28
29     public static void main(String[] args) {
30         String[] columnNames = {
31             "ID", "Cliente/Servidor", "Modelo Procesador", "Velocidad (GHz)",
32             "Nucleos", "Disco Duro Total (GB)", "SO", "CPU Libre (%)",
33             "Memoria Libre (GB)", "Ancho Banda Libre (%)", "Disco Libre (GB)", "Estado"
34         };
35     }
```

```

36 model = new DefaultTableModel(columnNames, 0) {
37     @Override
38     public boolean isCellEditable(int row, int column) {
39         return false;
40     }
41 };
42 table = new JTable(model);
43
44 // Configurar el frame
45 JFrame frame = new JFrame("Servidor");
46 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
47 frame.add(new JScrollPane(table));
48 frame.setSize(1200, 400);
49 frame.setVisible(true);
50
51 // Configurar el servidor con un puerto estático
52 try (ServerSocket serverSocket = new ServerSocket(SERVER_PORT)) {
53     // Agregar datos del servidor
54     addServerData();
55
56
57     // Hilo para actualizar datos del servidor y ordenar filas
58     ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(1);
59     scheduler.scheduleAtFixedRate(() -> {
60         updateServerData();
61         updateClientsData();
62         sortTableByCpuFree();
63     }, 0, UPDATE_INTERVAL, TimeUnit.MILLISECONDS);
64
65     // Aceptar conexiones de clientes
66     while (true) {
67         Socket clientSocket = serverSocket.accept();
68         ClientHandler handler = new ClientHandler(clientSocket);
69         new Thread(handler).start();
70     }
71 } catch (Exception e) {
72     e.printStackTrace();
73 }

```

```

74     }
75
76     private static void addServerData() {
77         SystemInfo si = new SystemInfo();
78         HardwareAbstractionLayer hal = si.getHardware();
79         CentralProcessor processor = hal.getProcessor();
80         OperatingSystem os = si.getOperatingSystem();
81         List<HWDiskStore> diskStores = hal.getDiskStores();
82
83         String processorModel = processor.getProcessorIdentifier().getName();
84         double processorSpeed = processor.getProcessorIdentifier().getVendorFreq() / 1e9;
85         int cores = processor.getLogicalProcessorCount();
86         long diskSize = diskStores.get(0).getSize() / (1024 * 1024 * 1024);
87
88         model.addRow(new Object[]{
89             System.getProperty("user.name"),
90             "Servidor",
91             processorModel,
92             processorSpeed,
93             cores,
94             diskSize,
95             os.toString(),
96             0.0, // CPU Libre
97             0.0, // Memoria Libre
98             0.0, // Ancho de banda
99             0.0, // Disco libre
100            "CONECTADO"
101        });
102    }
103
104     private static void updateServerData() {
105         SystemInfo si = new SystemInfo();
106         HardwareAbstractionLayer hal = si.getHardware();
107         CentralProcessor processor = hal.getProcessor();
108         GlobalMemory memory = hal.getMemory();
109         List<HWDiskStore> diskStores = hal.getDiskStores();
110
111         // Calcular CPU libre
112         long[] prevTicks = processor.getSystemCpuLoadTicks();
113         try { Thread.sleep(1000); } catch (InterruptedException e) {}
114         double cpuFree = 100.0 - (processor.getSystemCpuLoadBetweenTicks(prevTicks) * 100);
115
116         // Calcular memoria libre
117         double freeMemory = memory.getAvailable() / (1024.0 * 1024 * 1024);
118
119         // Calcular ancho de banda libre (aproximado basado en la utilización de red)
120         final double bandwidthFree;
121         List<NetworkIF> networks = hal.getNetworkIFs();
122         if (networks.isEmpty()) {
123             NetworkIF net = networks.get(0);
124             net.updateAttributes();
125             long totalBytes = net.getBytesRecv() + net.getBytesSent();
126             bandwidthFree = 100.0 - (totalBytes % 100); // Simplificado para ejemplo
127         } else {
128             bandwidthFree = 100.0;
129         }
130
131         // Calcular espacio libre en disco
132         File rootDisk = new File("/");
133         double diskFree = rootDisk.getUsableSpace() / (1024.0 * 1024 * 1024); // Convertimos a GB
134
135         // Actualizar fila del servidor
136         SwingUtilities.invokeLater(() -> {
137             model.setValueAt(String.format("%.2f", cpuFree), 0, 7);
138             model.setValueAt(String.format("%.2f", freeMemory), 0, 8);
139             model.setValueAt(String.format("%.2f", bandwidthFree), 0, 9);
140             model.setValueAt(String.format("%.2f", diskFree), 0, 10);
141         });
142     }
143
144     private static void updateClientsData() {
145         // Crear una lista temporal de clientes a remover
146         List<String> clientsToRemove = new ArrayList<>();
147
148         // Verificar clientes desconectados

```

```

149     connectedClients.forEach((id, handler) -> {
150         if (!handler.isConnected()) {
151             updateClientStatus(id, "DESCONECTADO");
152             clientsToRemove.add(id);
153         }
154     });
155
156     // Remover clientes desconectados
157     clientsToRemove.forEach(connectedClients::remove);
158
159     // Actualizar la tabla solo para clientes conectados
160     for (int i = 0; i < model.getRowCount(); i++) {
161         String clientId = (String) model.getValueAt(i, 0);
162         String status = (String) model.getValueAt(i, 11);
163
164         // Si es el servidor, continuar con el siguiente
165         if (clientId.equals("SERVIDOR")) {
166             continue;
167         }
168
169         // Si el cliente está desconectado, mantener los valores pero en gris
170         if (status.equals("DESCONECTADO")) {
171             table.setRowSelectionInterval(i, i);
172             table.setSelectionBackground(Color.LIGHT_GRAY);
173         }
174     }
175 }
176
177 private static void updateClientStatus(String clientId, String status) {
178     for (int i = 0; i < model.getRowCount(); i++) {
179         if (model.getValueAt(i, 0).equals(clientId)) {
180             model.setValueAt(status, i, 11);
181
182             if (status.equals("DESCONECTADO")) {
183                 // Limpiar todos los campos excepto ID y tipo de cliente
184                 model.setValueAt("", i, 2); // Modelo Procesador
185                 model.setValueAt(0.0, i, 3); // Velocidad
186                 model.setValueAt(0, i, 4); // Núcleos
187                 model.setValueAt(0.0, i, 5); // Disco Duro Total
188                 model.setValueAt("", i, 6); // SO
189                 model.setValueAt(0.0, i, 7); // CPU Libre
190                 model.setValueAt(0.0, i, 8); // Memoria Libre
191                 model.setValueAt(0.0, i, 9); // Ancho Banda Libre
192                 model.setValueAt(0.0, i, 10); // Disco Libre
193
194                 // Cambiar color de fondo a gris
195                 table.setRowSelectionInterval(i, i);
196                 table.setSelectionBackground(Color.LIGHT_GRAY);
197             } else {
198                 table.clearSelection();
199                 table.setSelectionBackground(table.getBackground());
200             }
201             break;
202         }
203     }
204     sortTableByCpuFree();
205 }
206
207 private static void sortTableByCpuFree() {
208     TableRowSorter<DefaultTableModel> sorter = new TableRowSorter<>(model);
209
210     // Configurar comparador personalizado
211     sorter.setComparator(7, (value1, value2) -> {
212         int row1 = -1, row2 = -1;
213
214         // Encontrar las filas correspondientes a los valores
215         for (int i = 0; i < model.getRowCount(); i++) {
216             if (model.getValueAt(i, 7).equals(value1)) row1 = i;
217             if (model.getValueAt(i, 7).equals(value2)) row2 = i;
218         }
219
220         // Obtener estados de conexión
221         String status1 = model.getValueAt(row1, 11).toString();
222         String status2 = model.getValueAt(row2, 11).toString();
223
224         // Si uno está desconectado y el otro no, el desconectado va al final

```

```

225         if (status1.equals("DESCONECTADO") && !status2.equals("DESCONECTADO")) return 1;
226         if (!status1.equals("DESCONECTADO") && status2.equals("DESCONECTADO")) return -1;
227
228         // Si ambos están desconectados, mantener el orden actual
229         if (status1.equals("DESCONECTADO") && status2.equals("DESCONECTADO")) {
230             return row1 - row2;
231         }
232
233         // Si ambos están conectados, ordenar por CPU libre
234         try {
235             double cpu1 = Double.parseDouble(value1.toString());
236             double cpu2 = Double.parseDouble(value2.toString());
237             return Double.compare(cpu2, cpu1); // Orden descendente
238         } catch (NumberFormatException e) {
239             return 0;
240         }
241     });
242
243     table.setRowSorter(sorter);
244
245     // Aplicar orden descendente en la columna 7 (CPU Libre)
246     List<RowSorter.SortKey> sortKeys = new ArrayList<>();
247     sortKeys.add(new RowSorter.SortKey(7, SortOrder.ASCENDING));
248     sorter.setSortKeys(sortKeys);
249     sorter.sort();
250 }
251
252 static class ClientHandler implements Runnable {
253     private final Socket socket;
254     private String clientId;
255     private volatile boolean connected = true;
256     private ObjectInputStream ois;
257
258     public ClientHandler(Socket socket) {
259         this.socket = socket;
260     }
261
262     @Override
263     public void run() {
264         try {
265             ois = new ObjectInputStream(socket.getInputStream());
266             // Esperar hasta que se reciba el ID del cliente
267             Object receivedObject = ois.readObject();
268             if (receivedObject instanceof Map) {
269                 Map<String, Object> clientSpecs = (Map<String, Object>) receivedObject;
270                 clientId = (String) clientSpecs.get("ID");
271                 // Agregar al mapa de clientes conectados
272                 connectedClients.put(clientId, this);
273
274                 while (connected) {
275                     receivedObject = ois.readObject();
276                     if (receivedObject instanceof Map) {
277                         clientSpecs = (Map<String, Object>) receivedObject;
278                         handleClientData(clientSpecs);
279                     }
280                 }
281             }
282         } catch (EOFException e) {
283             System.out.println("Cliente desconectado: " + clientId);
284             connected = false;
285         } catch (IOException e) {
286             System.out.println("Error de conexión con el cliente: " + clientId);
287             connected = false;
288         } catch (ClassNotFoundException e) {
289             e.printStackTrace();
290         } finally {
291             try {
292                 if (ois != null) {
293                     ois.close();
294                 }
295                 socket.close();
296             } catch (IOException e) {
297                 e.printStackTrace();
298             }
299         }

```



```

300         // Actualizar el estado del cliente en el servidor
301         if (clientId != null) {
302             connected = false;
303             updateClientStatus(clientId, "DESCONECTADO");
304         }
305     }
306 }
307
308 private void handleClientData(final Map<String, Object> clientSpecs) {
309     SwingUtilities.invokeLater(() -> {
310         boolean clientExists = false;
311         String clientId = (String) clientSpecs.get("ID");
312
313         for (int i = 0; i < model.getRowCount(); i++) {
314             if (model.getValueAt(i, 0).equals(clientId)) {
315                 // Si el cliente existe pero estaba desconectado, actualizar su estado
316                 if (model.getValueAt(i, 11).equals("DESCONECTADO")) {
317                     model.setValueAt("CONECTADO", i, 11);
318                     table.clearSelection();
319                     table.setSelectionBackground(table.getBackground());
320                 }
321                 updateRowData(i, clientSpecs);
322                 clientExists = true;
323                 break;
324             }
325         }
326
327         if (!clientExists) {
328             addNewClient(clientSpecs);
329         }
330
331         // Forzar reordenamiento después de actualizar
332         sortTableByCpuFree();
333     });
334 }
335
336 private void updateRowData(int row, Map<String, Object> specs) {
337     // Solo actualizar si el cliente está conectado
338     if (model.getValueAt(row, 11).equals("CONECTADO")) {
339         model.setValueAt(specs.get("Modelo del Procesador"), row, 2);
340         model.setValueAt(specs.get("Velocidad del Procesador (GHz)"), row, 3);
341         model.setValueAt(specs.get("Nucleos"), row, 4);
342         model.setValueAt(specs.get("Capacidad del Disco Duro (GB)"), row, 5);
343         model.setValueAt(specs.get("Version del Sistema Operativo"), row, 6);
344         model.setValueAt(specs.get("CPU Libre"), row, 7);
345         model.setValueAt(specs.get("Memoria Libre"), row, 8);
346         model.setValueAt(specs.get("Ancho de Banda Libre"), row, 9);
347         model.setValueAt(specs.get("Disco Libre"), row, 10);
348     }
349 }
350
351 private void addNewClient(Map<String, Object> specs) {
352     model.addRow(new Object[]{
353         specs.get("ID"),
354         "Cliente",
355         specs.get("Modelo del Procesador"),
356         specs.get("Velocidad del Procesador (GHz)"),
357         specs.get("Nucleos"),
358         specs.get("Capacidad del Disco Duro (GB)"),
359         specs.get("Version del Sistema Operativo"),
360         specs.get("CPU Libre"),
361         specs.get("Memoria Libre"),
362         specs.get("Ancho de Banda Libre"),
363         specs.get("Disco Libre"),
364         "CONECTADO"
365     });
366 }
367
368 public boolean isConnected() {
369     return connected;
370 }
371 }
372 }

```

CLIENTE

```
1 package Ej;
2
3 import oshi.SystemInfo;
4 import oshi.hardware.*;
5 import oshi.software.os.OperatingSystem;
6
7 import javax.swing.*;
8 import javax.swing.table.DefaultTableModel;
9 import java.io.*;
10 import java.net.Socket;
11 import java.util.HashMap;
12 import java.util.List;
13 import java.util.Map;
14 import java.util.Scanner;
15
16
17 public class Client {
18     private static final int UPDATE_INTERVAL = 5000; // 5 segundos
19     private static final String SERVER_IP = "25.51.170.2"; // IP estática del servidor
20     private static final int SERVER_PORT = 8080; // Puerto estático
21
22     private static DefaultTableModel tableModel; // Modelo de la tabla
23     private static JTable table; // Tabla para mostrar los datos
24
25     public static void main(String[] args) {
26         // Crear la ventana de la interfaz gráfica
27         JFrame frame = new JFrame("Cliente");
28         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
29
30         // Definir las columnas de la tabla
31         String[] columnNames = {
32             "ID", "Modelo Procesador", "Velocidad (GHz)",
33             "Núcleos", "Disco Duro Total (GB)", "SO", "CPU Libre (%)",
34             "Memoria Libre (GB)", "Ancho Banda Libre (%)", "Disco Libre (GB)"
35         };
36
37         // Crear el modelo de la tabla
38         tableModel = new DefaultTableModel(columnNames, 0) {
39             @Override
40             public boolean isCellEditable(int row, int column) {
41                 return false; // Hacer que las celdas no sean editables
42             }
43         };
44         table = new JTable(tableModel);
45
46         // Agregar la tabla a un JScrollPane para hacerla desplazable
47         frame.add(new JScrollPane(table));
48         frame.setSize(1200, 78); // Tamaño de la ventana
49         frame.setLocationRelativeTo(null); // Centrar la ventana en la pantalla
50         frame.setVisible(true); // Mostrar la ventana
51
52         try (Scanner scanner = new Scanner(System.in)) {
53             // Usamos IP estática y puerto estático
54             String serverIp = SERVER_IP;
55             int port = SERVER_PORT;
56
57             // Recuperar el ID guardado o solicitar uno nuevo
58             String clientId = getUserId();
59
60
61             Socket socket = new Socket(serverIp, port);
62             ObjectOutputStream oos = new ObjectOutputStream(socket.getOutputStream());
63
64             // Agregar fila "Calculando..." a la tabla
65             Object[] calculatingRow = new Object[columnNames.length];
66             for (int i = 0; i < columnNames.length; i++) {
67                 calculatingRow[i] = "Calculando..."; // Asignar "Calculando..." a todas las columnas
68             }
69             tableModel.addRow(calculatingRow);
70
71
72             // Hilo para enviar actualizaciones periódicas
73             Thread updateThread = new Thread(() -> {
74                 try {
```

```

75         while (!Thread.interrupted()) {
76             Map<String, Object> specs = getSystemSpecs(clientId);
77             oos.writeObject(specs);
78             oos.reset(); // Limpiar cache del stream
79             updateTable(specs);
80             Thread.sleep(UPDATE_INTERVAL);
81         }
82     } catch (Exception e) {
83         System.out.println("Conexión perdida con el servidor.");
84     }
85 });
86
87 updateThread.start();
88
89 // Esperar entrada del usuario para terminar
90 System.out.println("\nPresiona ENTER para desconectarte");
91 scanner.nextLine();
92 updateThread.interrupt();
93 socket.close();
94
95 } catch (Exception e) {
96     e.printStackTrace();
97 }
98 }
99
100
101
102 private static String getUserId() {
103     // Obtener el nombre del usuario actual del sistema
104     return System.getProperty("user.name");
105 }
106
107 private static Map<String, Object> getSystemSpecs(String clientId) {
108     SystemInfo si = new SystemInfo();
109     HardwareAbstractionLayer hal = si.getHardware();
110     CentralProcessor processor = hal.getProcessor();
111     GlobalMemory memory = hal.getMemory();
112     List<HWDiskStore> diskStores = hal.getDiskStores();

```

```

113 List<NetworkIF> networks = hal.getNetworkIFs();
114 OperatingSystem os = si.getOperatingSystem();
115
116 long[] prevTicks = processor.getSystemCpuLoadTicks();
117 try { Thread.sleep(1000); } catch (InterruptedException e) { }
118 double cpuFree = 100.0 - (processor.getSystemCpuLoadBetweenTicks(prevTicks) * 100);
119
120 double freeMemory = memory.getAvailable() / (1024.0 * 1024 * 1024);
121
122 double bandwidthFree = 100.0;
123 if (!networks.isEmpty()) {
124     NetworkIF net = networks.get(0);
125     net.updateAttributes();
126     long totalBytes = net.getBytesRecv() + net.getBytesSent();
127     bandwidthFree = 100.0 - (totalBytes % 100);
128 }
129
130 File rootDisk = new File("/");
131 double diskFree = rootDisk.getUsableSpace() / (1024.0 * 1024 * 1024);
132
133 Map<String, Object> specs = new HashMap<>();
134 specs.put("ID", clientId);
135 specs.put("Modelo del Procesador", processor.getProcessorIdentifier().getName());
136 specs.put("Velocidad del Procesador (GHz)", processor.getProcessorIdentifier().getVendorFreq() / 1e9);
137 specs.put("Nucleos", processor.getLogicalProcessorCount());
138 specs.put("Capacidad del Disco Duro (GB)", diskStores.get(0).getSize() / (1024 * 1024 * 1024));
139 specs.put("Version del Sistema Operativo", os.toString());
140 specs.put("CPU Libre", String.format("%.2f", cpuFree));
141 specs.put("Memoria Libre", String.format("%.2f", freeMemory));
142 specs.put("Ancho de Banda Libre", String.format("%.2f", bandwidthFree));
143 specs.put("Disco Libre", String.format("%.2f", diskFree));
144
145 return specs;
146 }
147 private static void updateTable(Map<String, Object> specs) {
148     // Limpiar los datos actuales en la tabla
149     tableModel.setRowCount(0);
150

```

```

151     // Crear una fila que coincida con las columnas de la tabla
152     Object[] rowData = new Object[10]; // Porque tienes 10 columnas
153
154     // Asignar valores a las columnas específicas según el mapa 'specs'
155     rowData[0] = specs.get("ID"); // ID
156     rowData[1] = specs.get("Modelo Procesador"); // Modelo Procesador
157     rowData[2] = specs.get("Velocidad (GHz)"); // Velocidad (GHz)
158     rowData[3] = specs.get("Nucleos"); // Nucleos
159     rowData[4] = specs.get("Disco Duro Total (GB)"); // Disco Duro Total (GB)
160     rowData[5] = specs.get("SO"); // SO
161     rowData[6] = specs.get("CPU Libre (%)"); // CPU Libre (%)
162     rowData[7] = specs.get("Memoria Libre (GB)"); // Memoria Libre (GB)
163     rowData[8] = specs.get("Ancho Banda Libre (%)"); // Ancho Banda Libre (%)
164     rowData[9] = specs.get("Disco Libre (GB)"); // Disco Libre (GB)
165
166     // Agregar la fila a la tabla
167     tableModel.addRow(rowData);
168 }
169 }
170

```

CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA (CLIENTE Y SERVIDOR)

SERVIDOR

Monitor de Sistemas

ID	Cliente/Servidor	Modelo Procesa...	Velocidad (GHz)	Nucleos	Disco Duro Total...	SO	CPU Libre (%) ▲	Memoria Libre (...)	Ancho Banda Lib.	Disco Libre (GB)	Estado
andym	Cliente	AMD Ryzen 5 34...	2.096	8	238	Microsoft Windo...	97.79	0.55	96.00	7.39	CONECTADO
MELI	Servidor	12th Gen Intel(R)...	2.688	16	476	Microsoft Windo...	97.13	3.09	100.00	340.02	CONECTADO
heidi	Cliente	11th Gen Intel(R)...	2.688	12	476	Microsoft Windo...	95.22	3.85	100.00	67.00	CONECTADO
alexi	Cliente	13th Gen Intel(R)...	2.611	12	476	Microsoft Windo...	92.18	6.62	4.00	357.46	CONECTADO
aleja	Cliente	11th Gen Intel(R)...	3.302	8	476	Microsoft Windo...	82.74	8.58	100.00	1.88	CONECTADO

CLIENTE

Cliente										
ID	Modelo Procesador	Velocidad (GHz)	Nucleos	Disco Duro Total (G..	SO	CPU Libre (%)	Memoria Libre (GB)	Ancho Banda Libre (...)	Disco Libre (GB)	
heidi	11th Gen Intel(R) Co	2.688	12	476	Microsoft Windows 1	97.98	3.91	100.00	66.92	

28 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

Presiona ENTER para desconectarte

22:51:57.144 [Thread-0] WARN oshi.util.platform.windows.PerfCounterWildcardQuery - Disabling further attempts to query PhysicalDisk.

22:51:57.483 [Thread-0] DEBUG oshi.software.os.windows.WindowsOperatingSystem - Debug privileges not enabled.

22:51:57.493 [Thread-0] WARN oshi.util.platform.windows.PerfCounterWildcardQuery - Disabling further attempts to query Processor Information.

Conclusiones

1952947 - Heidi Pamela Martínez Martínez

Conclusión

Heidi Pamela Martínez Martínez 1952947

Aprendí como manejar muchos procesos simultáneamente en cliente-servidor, separando tareas como la actualización de la interfaz gráfica y el envío de datos, utilizando hilos para evitar que se bloqueen entre sí como el problema que tuvimos. También quise a ratos en función de cada desempeño.

Este conocimiento me sirve para desarrollar aplicaciones de monitoreo de sistemas y optimizar el uso de recursos de redes gracias a los errores que tuvimos conociendo, así, como resolviéndolos.

Conclusión - Andrea Monserrat Méndez Flores
1845517

Para esta tarea fue necesario obtener las características de nuestras dispositivos como ya se había realizado anteriormente en prácticas anteriores, se pidió implementar un algoritmo de ranking para que una vez obtenidas las características, el algoritmo pudiera identificar el más adecuado de acuerdo a lo pedido en la tarea. Al principio el código no se ejecutaba de manera correcta y arrojaba errores pero luego se logró solucionar.

Alexia Hernandez Andrade 2109147

Conclusión Tarea Practica #6

En conclusión esta tarea practica nos permitio implementar un algoritmo de rankeo basado en las características estaticas y dinamicas de las computadoras conectadas al servidor. Este proceso fue clave para determinar cual dispositivo tenia un mejor rendimiento y conectividad. Sin embargo fue un poco complicado por los errores que arrojaba el codigo especialmente al intentar agregar esa nueva Funcionalidad. Es por eso que esta actividad fue fundamental para nuestro aprendizaje.

Alejandro Sauqué Pérez #2004428

Conclusión Individual Sis. Dist. y Paralelos

Personalmente, aprendí de la importancia del ranqueo efectivo para poder evaluar la calidad de conexión de los modos de una red. Este proceso me permite entender de manera objetiva como funcionan los sistemas reales que a primera vista llegan a parecer demasiado complejos (aunque quizás sí lo son). Me parece que la habilidad para identificar la mejor conexión es fundamental. Si hubo problemas y dificultades para hacer esta práctica, pero sí permitió un aprendizaje más profundo.

Conclusion

2077884

En conclusión el sistema de monitoreo implementado permite una comunicación efectiva entre cliente y servidor, permitiendo la visualización en tiempo real en diversas métricas de rendimiento de ambos dispositivos.

Se utiliza una arquitectura cliente-servidor en Java con una o con el soporte de la librería oshi para la obtención de datos de hardware y sistema operativo. Este enfoque facilita la supervisión del estado de los dispositivos conectados, lo cual es útil para el análisis de rendimiento y la gestión de recursos.

Y a través del ranking permite una visualización rápida de los recursos en uso y ayuda a identificar posibles cuellos de botella.

Aun hay algunos problemas en cuestión de cambiar de red de servidor pero se intentan solucionar.